**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# LESSON 317 · Paper 2 extended response challenge

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What is a precondition?

Q2. What is a loop invariant?

Q3. What does O(1) auxiliary space mean?

Q4. For [1,1,2,3,3], count globally distinct values and consecutive runs.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

precondition — Required assumption about input.

invariant — Property maintained by a loop.

auxiliary space — Memory used beyond input.

hash set — Structure for expected efficient membership tests.

## New learning

Step 1 — State the sorted-input precondition and any edge cases.

Step 2 — Trace changing values with a loop invariant and correct bounds.

Step 3 — Explain why the selected algorithm is efficient and when another is preferable.

## Worked example

Trace UniqueCount for [1,1,2,3,3].

1. Explain the worked method — The loop starts at index1 and compares each item with its predecessor.

# We Do · Practise with guidance

Trace UniqueCount for [4,4,4,9,10,10].

```
Return3; groups 4,9,10.
```

Explain the worked method

The comparison counts new groups, not every repeated value.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Write pseudocode to count distinct values in a sorted array, including empty input. [8]

### Question 2

Trace [2,2,5,5,6,9,9] and give time/space complexities. [9]

### Question 3

Evaluate adjacent counting versus a hash set when the data is unsorted and contains a million elements. [9]

## Exit Ticket · Answer before checking

An adjacent comparison algorithm for sorted input works identically on unsorted input.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. Requirement that must be satisfied for correctness.

Q2. Property maintained as iterations progress.

Q3. Additional memory remains constant with input size.

Q4. Three distinct values; three consecutive runs.

### You Do pages · Independent questions

Q1. Mark scheme [8]: test for an empty array [1] and return 0 [1]; initialise count to 1 [1]; loop from index 1 to the last valid index [1]; compare adjacent values with != [2]; increment on change [1]; return count [1]. An equivalent clear pseudocode solution earns the same marks.

```
CODE:
function countDistinct(values)
if values.length == 0 then
return 0
endif
count = 1
for i = 1 to values.length - 1
if values[i] != values[i-1] then
count = count + 1
endif
next i
return count
endfunction
```

Q2. Start1 [1]; change to5→2 [2]; to6→3 [2]; to9→4 [2]; O(n) time [1] and O(1) additional space [1].

Q3. Indicative content: adjacent counting requires sorted input; after sorting, counting is O(n) time with O(1) extra counting state; comparison sorting often costs O(n log n), whereas a hash set can count distinct values in expected O(n) time with O(n) extra space and possible collision overhead. Practice marking (authored guidance, not an OCR mark scheme): 7–9 = accurate, sustained evaluation of both





BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

options, applied to this scenario with a supported judgement; 4–6 = relevant comparison with some development and a partly justified decision; 1–3 = basic statements, limited context or unsupported decision; 0 = no creditworthy response. Use best-fit and credit equivalent valid arguments.
Model evaluation: The adjacent-comparison algorithm is excellent when the million values are already sorted: each value is inspected once, so time is O(n) and only a counter is needed. On unsorted data, identical values may be separated and it produces the wrong count unless the values are first sorted. Comparison sorting typically adds O(n log n) time, with space depending on the sorting method. A hash set processes an unsorted stream in expected O(n) time by recording each distinct value, but requires up to O(n) additional memory and performance depends on hashing and collisions. With sufficient memory and a one-off unsorted input, I would choose a hash set; for an already sorted array or strict memory constraints, adjacent counting is preferable.

### Exit Ticket · Model answer

False. Separated duplicate values can be counted repeatedly.

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**

Do Now Recall previous learning | I Do Read new knowledge | We Do Practise with guidance | You Do Work independently | Review & Improve Check my understanding